Adaptive Structured Pruning of Convolutional Neural Networks for Time Series Classification

Javidan Abdullayev, Maxime Devanne, Cyril Meyer, Ali Ismail-Fawaz, Jonathan Weber, Germain Forestier

IRIMAS, Université de Haute-Alsace

March 7, 2026

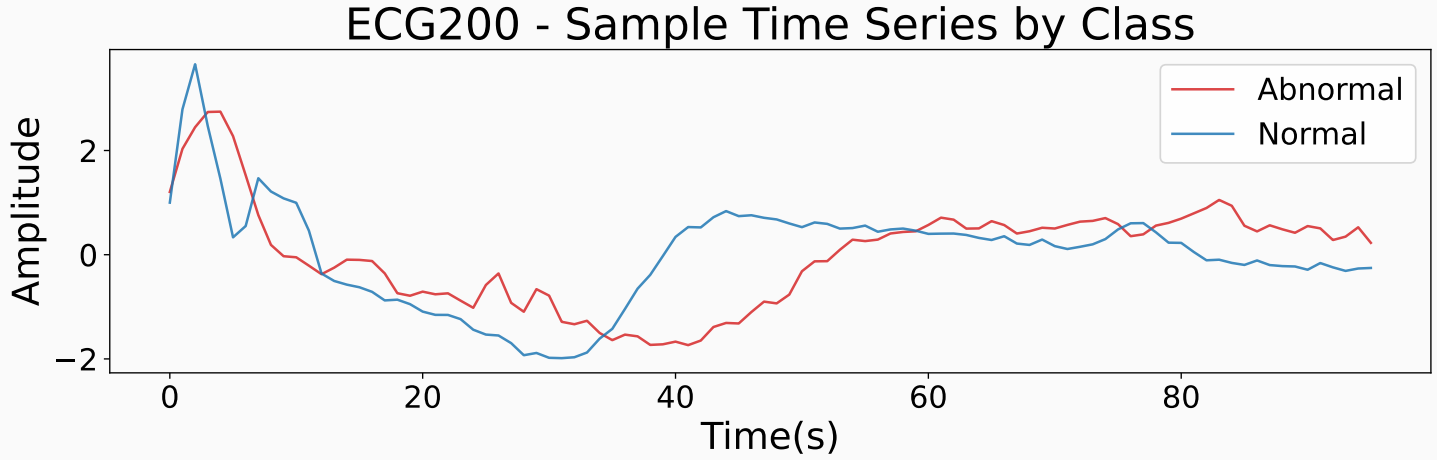
1. Introduction and Literature Review
2. Proposed Method
3. Experimental Evaluation

1 Introduction and Literature Review

2 Proposed Method

3 Experimental Evaluation

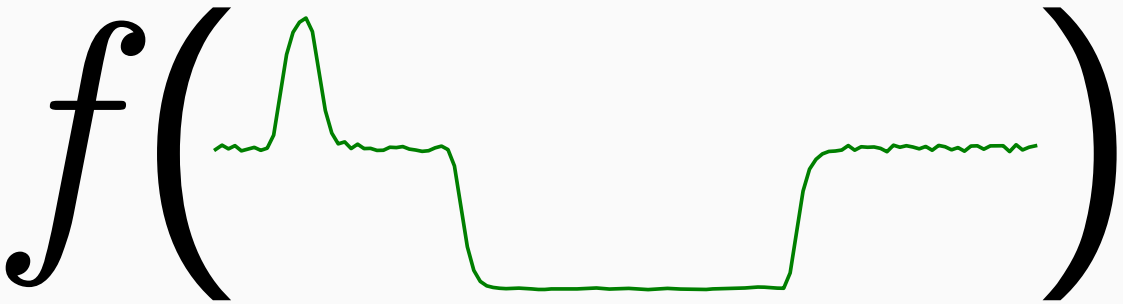
Time Series Data



Dau, Hoang Anh, et al. "The UCR time series archive." IEEE/CAA Journal of Automatica Sinica 6.6 (2019): 1293-1305.

Time Series Classification (TSC)

Time Series Classification Task



- f is a classifier
- f assigns a label y to an input sample
- $y \in \{0, \ldots, C-1\}$, with the C number of classes

Deep Learning Approaches for TSC

- Deep Learning (DL) for TSC

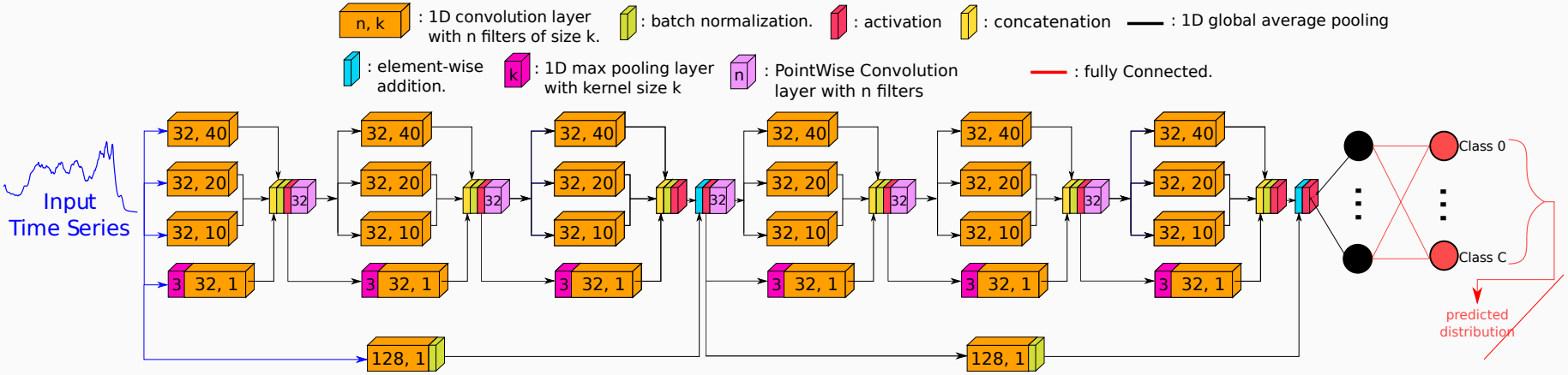
  Best performing DL approaches for TSC based on Convolutional Neural Networks

Middlehurst et al. "Bake off redux." Data Mining and Knowledge Discovery 38.4 (2024).

Deep Learning Approaches for TSC

- Deep Learning (DL) for TSC

Best performing DL approaches for TSC based on Convolutional Neural Networks



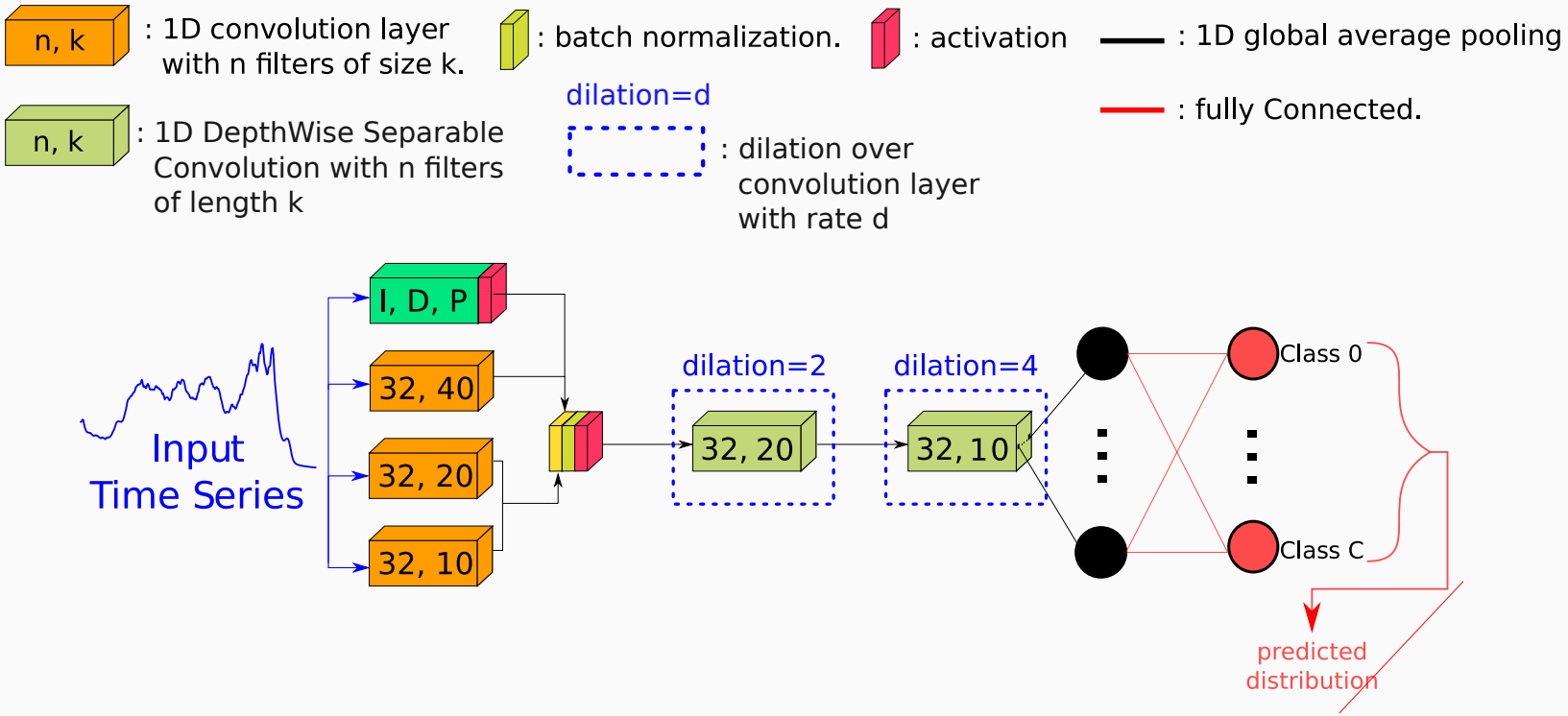
InceptionTime Architecture

Ismail Fawaz, Hassan, et al. "Inceptiontime: Finding alexnet for time series classification." Data mining and knowledge discovery 34.6 (2020): 1936-1962.

Deep Learning Approaches for TSC

▷ Deep Learning (DL) for TSC

Best performing DL approaches for TSC based on Convolutional Neural Networks



Lightweight InceptionTime Ensemble (LITE)

Ismail-Fawaz, Ali, et al. "Lite: Light inception with boosting techniques for time series classification." 2023 IEEE 10th International Conference on Data Science and Advanced Analytics (DSAA). IEEE, 2023.

Motivation

- TSC benchmarks show high dataset diversity:
  - 128 UCR datasets with varying complexity
  - Time series length: 24 – 2,844 | Classes: 2 – 60 | Samples: 16 – 8,926

Motivation

- TSC benchmarks show high dataset diversity:
  - 128 UCR datasets with varying complexity
  - Time series length: 24 – 2,844 | Classes: 2 – 60 | Samples: 16 – 8,926
- Current SOTA models apply uniformly across all datasets:
  - InceptionTime, H-InceptionTime, LITETime
  - Same model size regardless of dataset complexity
  - Unnecessarily large for simple tasks

Motivation

- TSC benchmarks show high dataset diversity:
  - 128 UCR datasets with varying complexity
  - Time series length: 24 – 2,844 | Classes: 2 – 60 | Samples: 16 – 8,926
- Current SOTA models apply uniformly across all datasets:
  - InceptionTime, H-InceptionTime, LITETime
  - Same model size regardless of dataset complexity
  - Unnecessarily large for simple tasks
- **Research Question:**

  Can we automatically adapt model complexity to dataset difficulty without manual tuning of compression ratios?

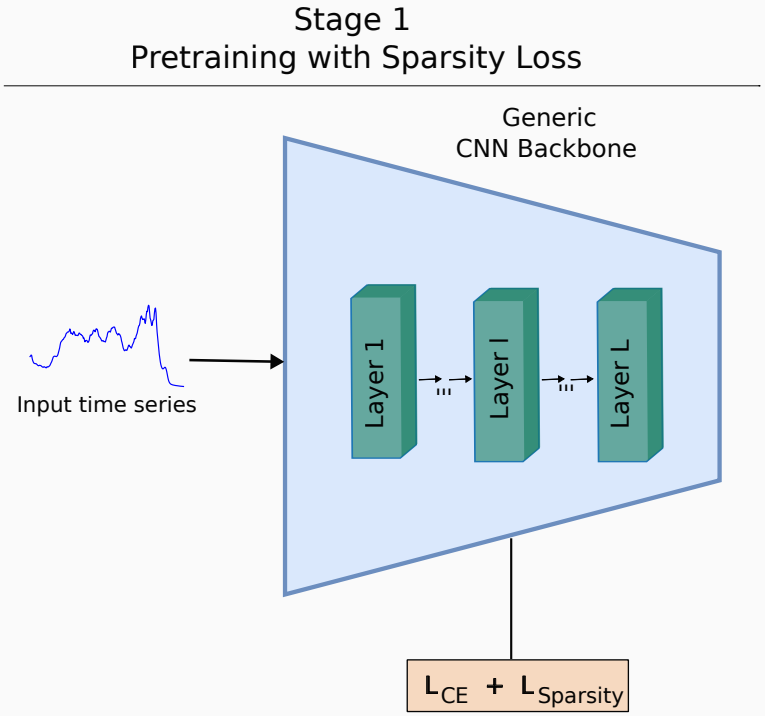
1 Introduction and Literature Review

2 Proposed Method

3 Experimental Evaluation

Proposed Approach

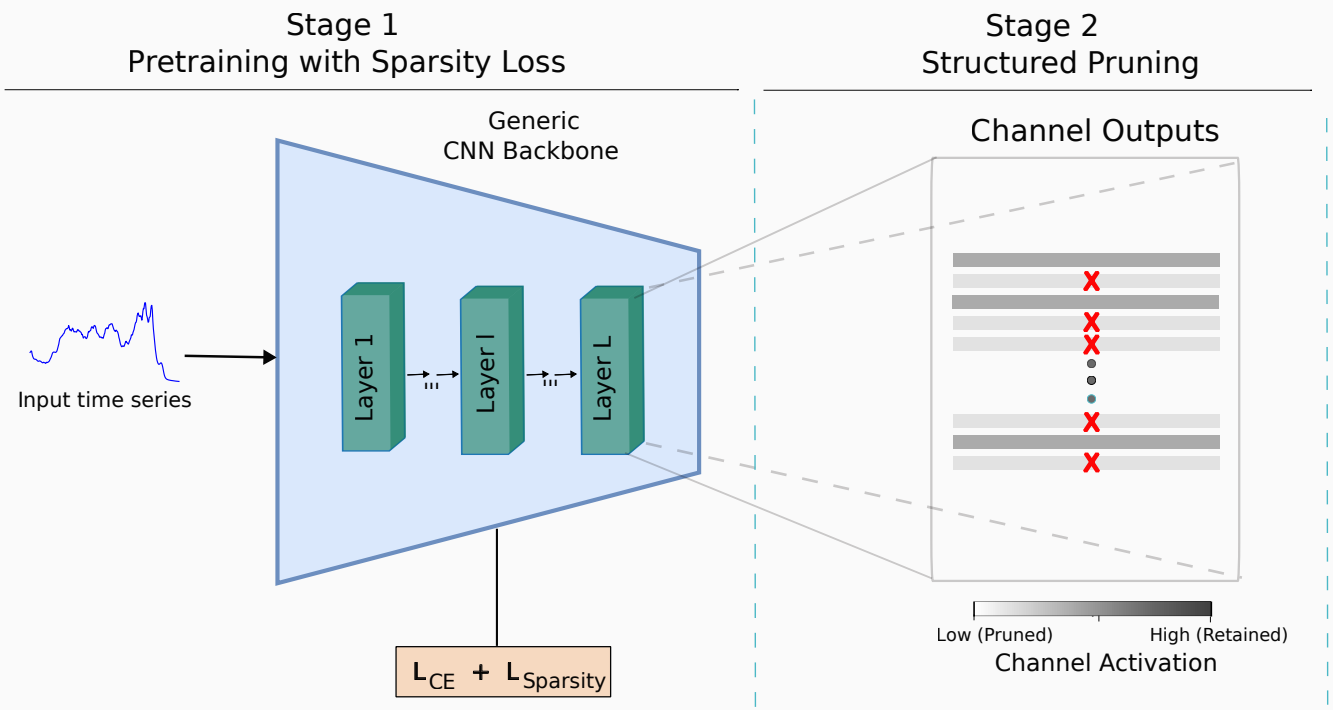
Overview of our Dynamic Structured Pruning Approach



▷ **Stage 1:** Pretraining with *Auxiliary* Sparsity Loss - Promotes channel-level sparsity

Proposed Approach

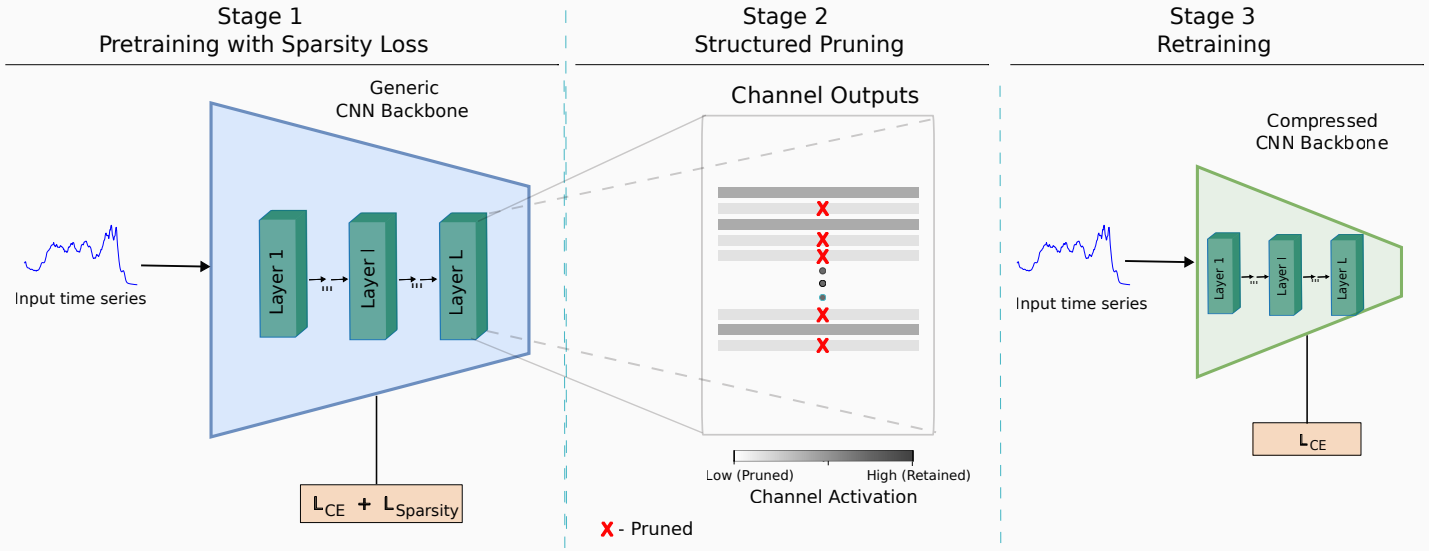
Overview of our Dynamic Structured Pruning Approach



- ▷ **Stage 1:** Pretraining with *Auxiliary* Sparsity Loss - Promotes channel-level sparsity
- ▷ **Stage 2:** Structured Pruning - Prune low-activation filters globally

Proposed Approach

Overview of our Dynamic Structured Pruning Approach

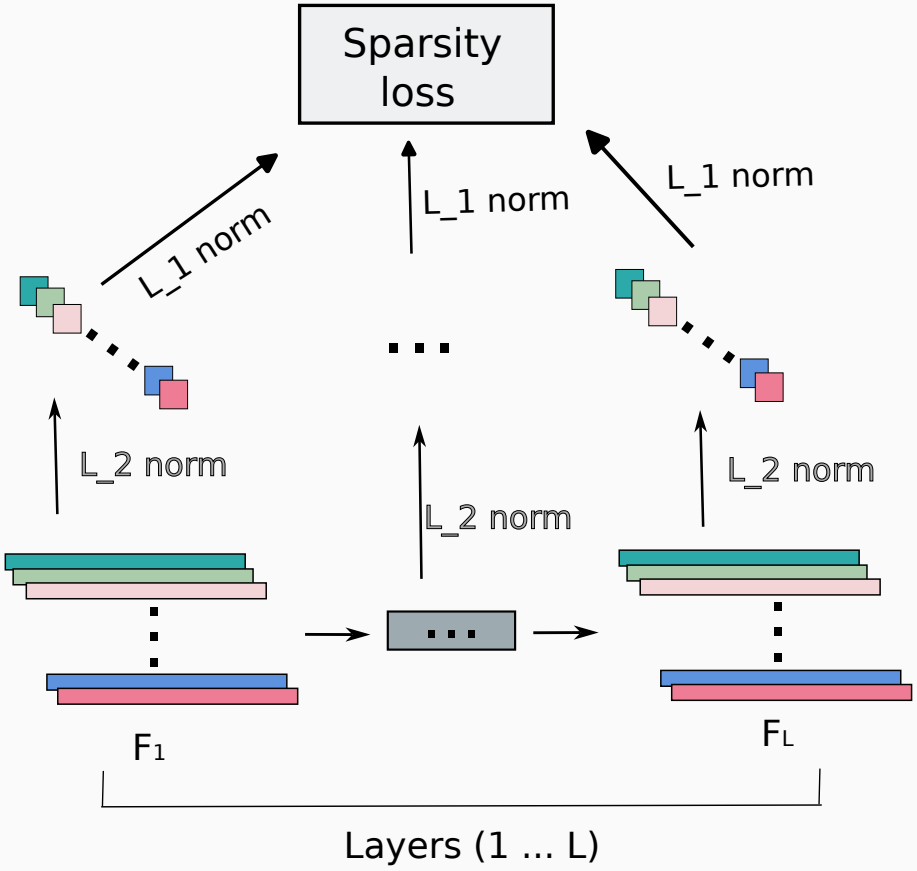


- ▷ **Stage 1:** Pretraining with *Auxiliary* Sparsity Loss - Promotes channel-level sparsity
- ▷ **Stage 2:** Structured Pruning - Prune low-activation filters globally
- ▷ **Stage 3:** Retraining - Maintains original accuracy

Stage 1: Pretraining with Sparsity Loss

Hierarchical Sparsity Loss:

$$\mathcal{L}_{\text{sparsity}} = \sum_{i=1}^{B} \sum_{j=1}^{C} \underbrace{\sqrt{\sum_{k=1}^{T} F[i,j,k]^2}}_{L_2 \text{ norm}}$$



Instance-wise sparsity loss induces channel-level sparsity

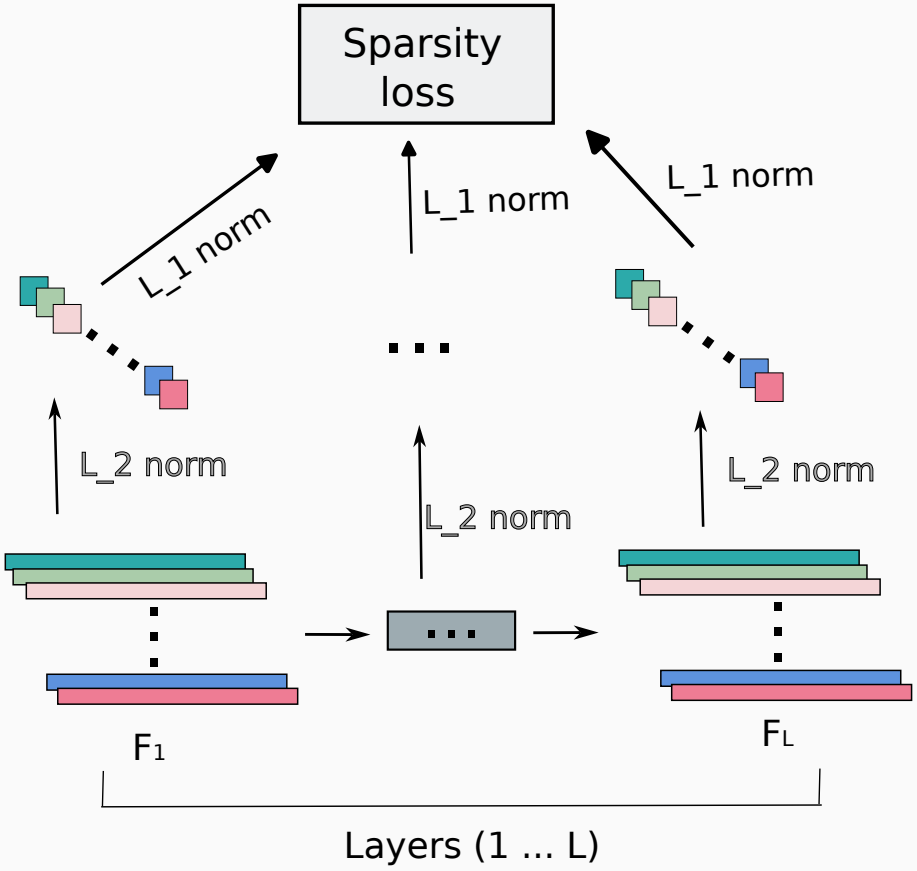
Stage 1: Pretraining with Sparsity Loss

Hierarchical Sparsity Loss:

$$\mathcal{L}_{\text{sparsity}} = \sum_{i=1}^{B} \sum_{j=1}^{C} \underbrace{\sqrt{\sum_{k=1}^{T} F[i,j,k]^2}}_{L_2 \text{ norm}}$$

How it works:

- L_2 norm: Measures channel energy
- L_1 norm: Enforces global sparsity
- Entire channels become inactive



Instance-wise sparsity loss induces channel-level sparsity

Stage 1: Pretraining with Sparsity Loss

Hierarchical Sparsity Loss:

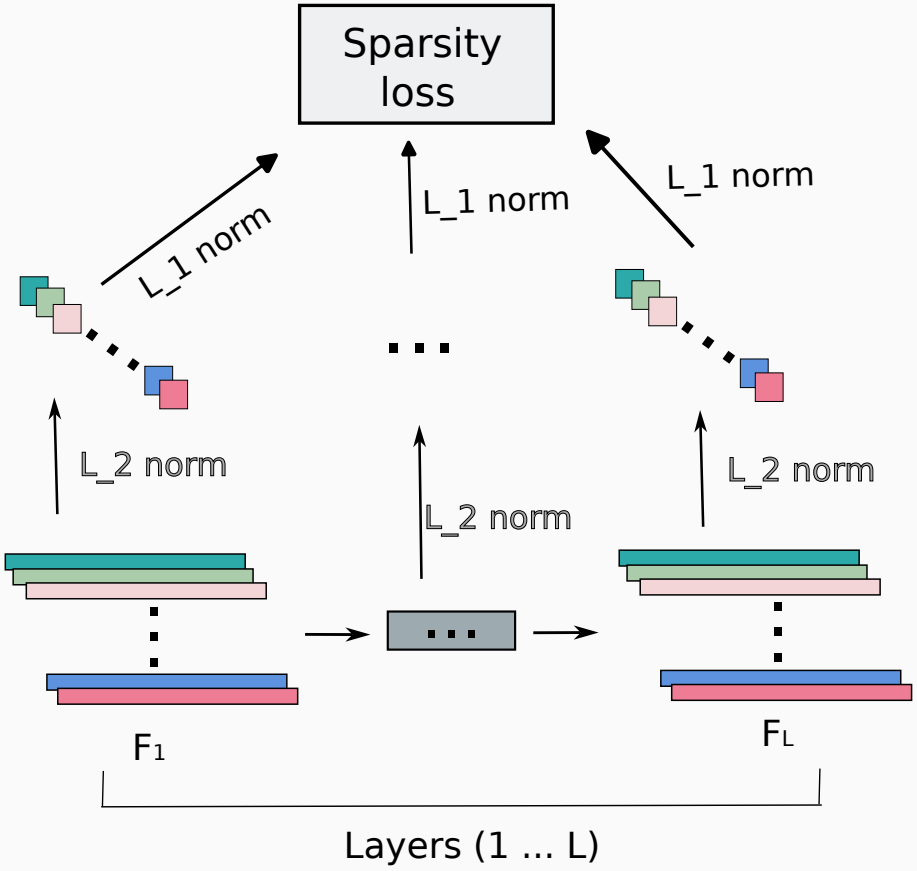
$$\mathcal{L}_{\text{sparsity}} = \sum_{i=1}^{B} \sum_{j=1}^{C} \underbrace{\sqrt{\sum_{k=1}^{T} F[i,j,k]^2}}_{L_2 \text{ norm}}$$

How it works:

- L_2 norm: Measures channel energy
- L_1 norm: Enforces global sparsity
- Entire channels become inactive

Training Objective:

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{CE} + \lambda \cdot \mathcal{L}_{\text{sparsity}}$$

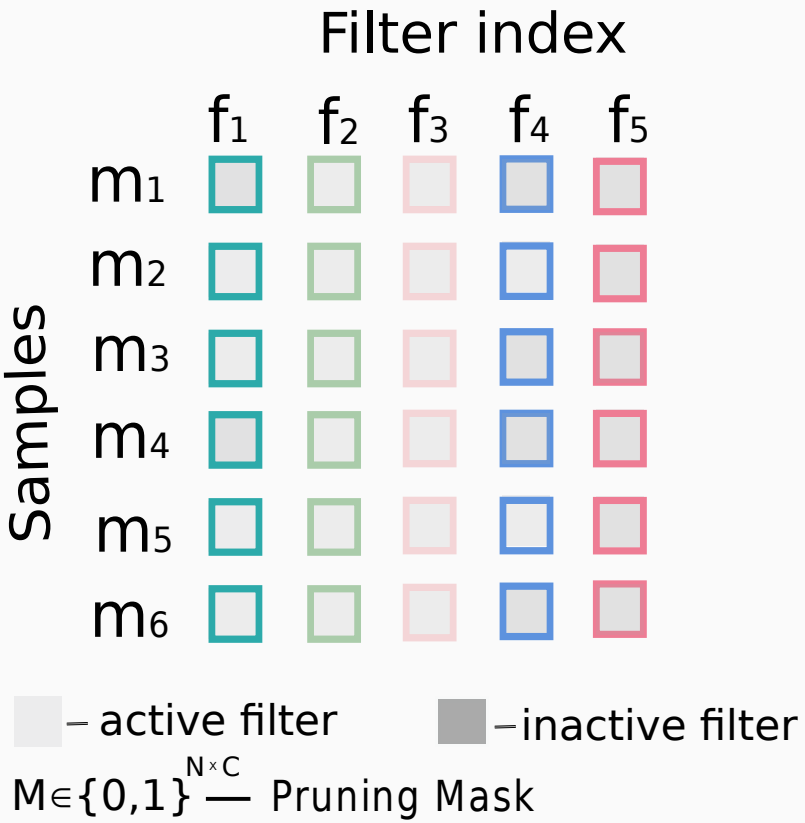


Instance-wise sparsity loss induces channel-level sparsity

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

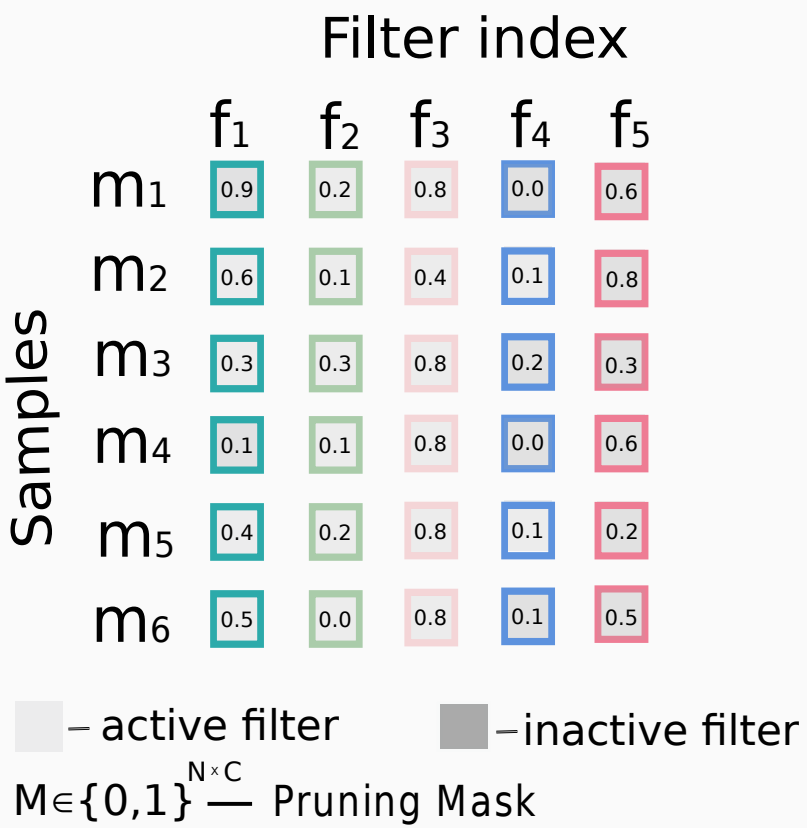


Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

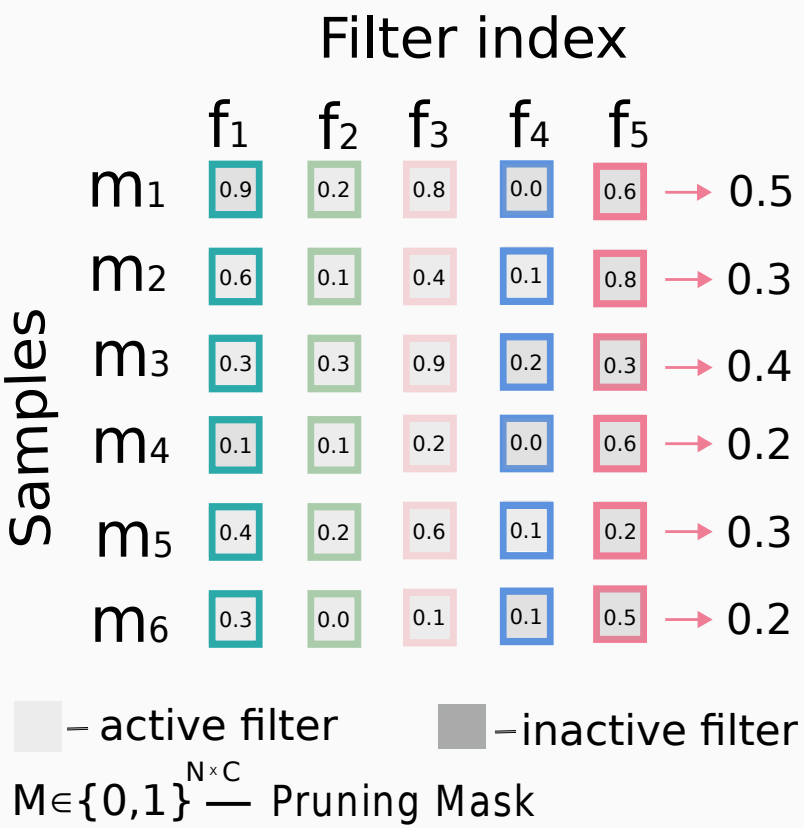
Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- ▷ Instance-wise threshold
- ▷ No manual pruning ratio



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

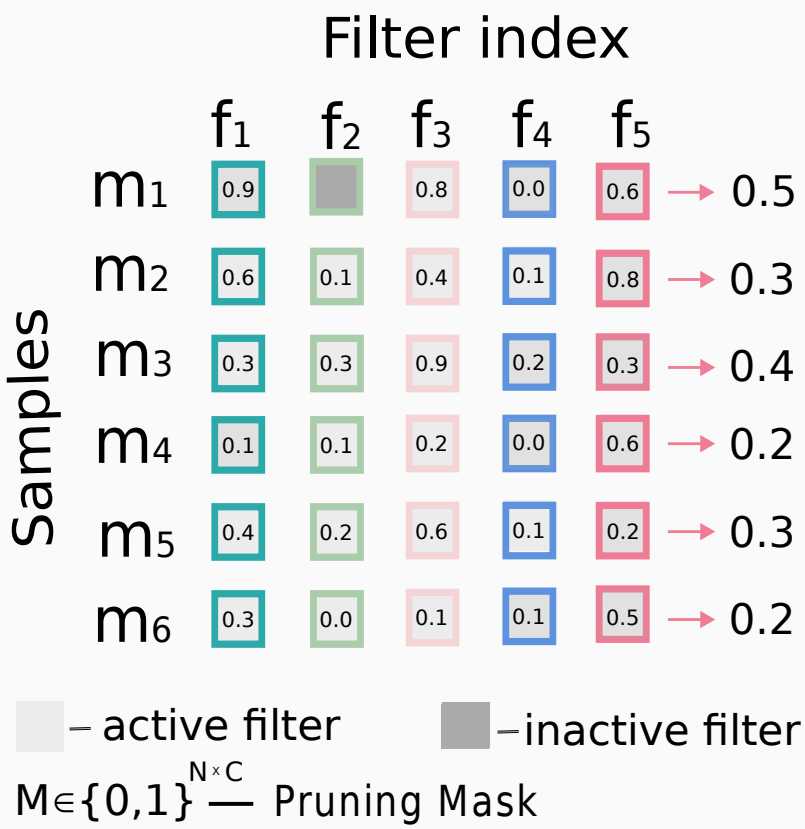
$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- Instance-wise threshold
- No manual pruning ratio

Global Consensus Pruning:

$$M[i,j] = \begin{cases} 1 & \text{if } s_{ij} \geq \tau_i \\ 0 & \text{otherwise} \end{cases}$$

- Prune only if inactive for ALL samples
- Preserves class-specific features



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

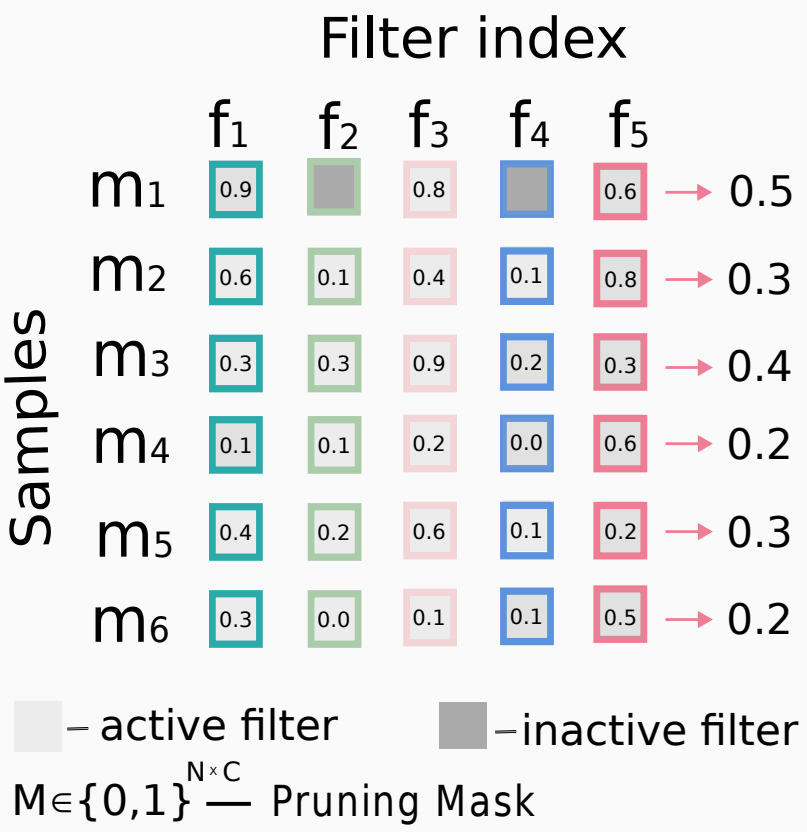
$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- ▷ Instance-wise threshold
- ▷ No manual pruning ratio

Global Consensus Pruning:

$$M[i,j] = \begin{cases} 1 & \text{if } s_{ij} \geq \tau_i \\ 0 & \text{otherwise} \end{cases}$$

- ▷ Prune only if inactive for ALL samples
- ▷ Preserves class-specific features



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

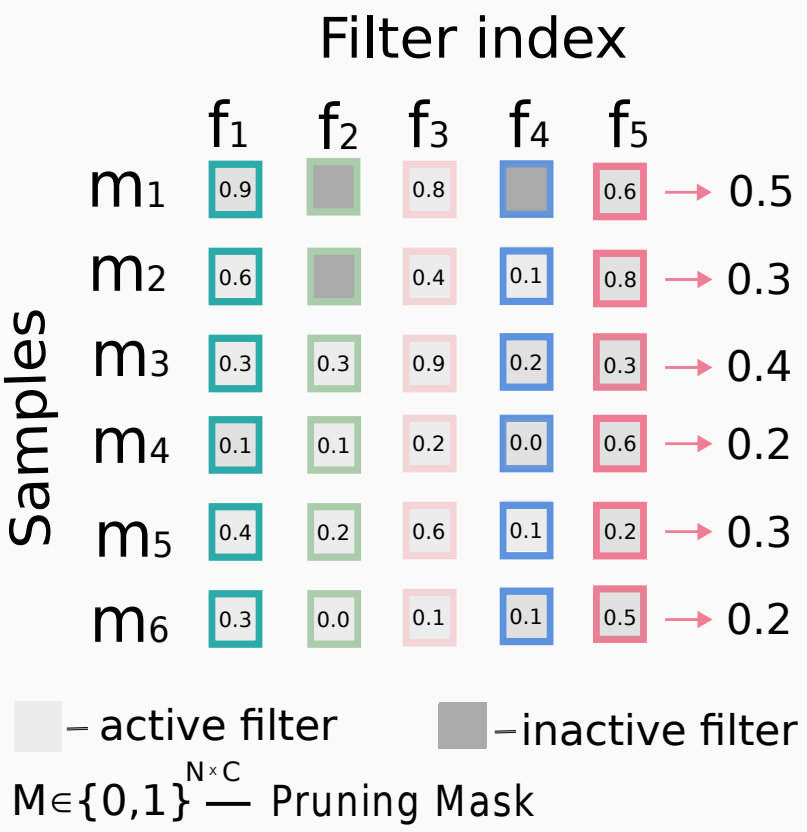
$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- ▷ Instance-wise threshold
- ▷ No manual pruning ratio

Global Consensus Pruning:

$$M[i,j] = \begin{cases} 1 & \text{if } s_{ij} \geq \tau_i \\ 0 & \text{otherwise} \end{cases}$$

- ▷ Prune only if inactive for ALL samples
- ▷ Preserves class-specific features



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

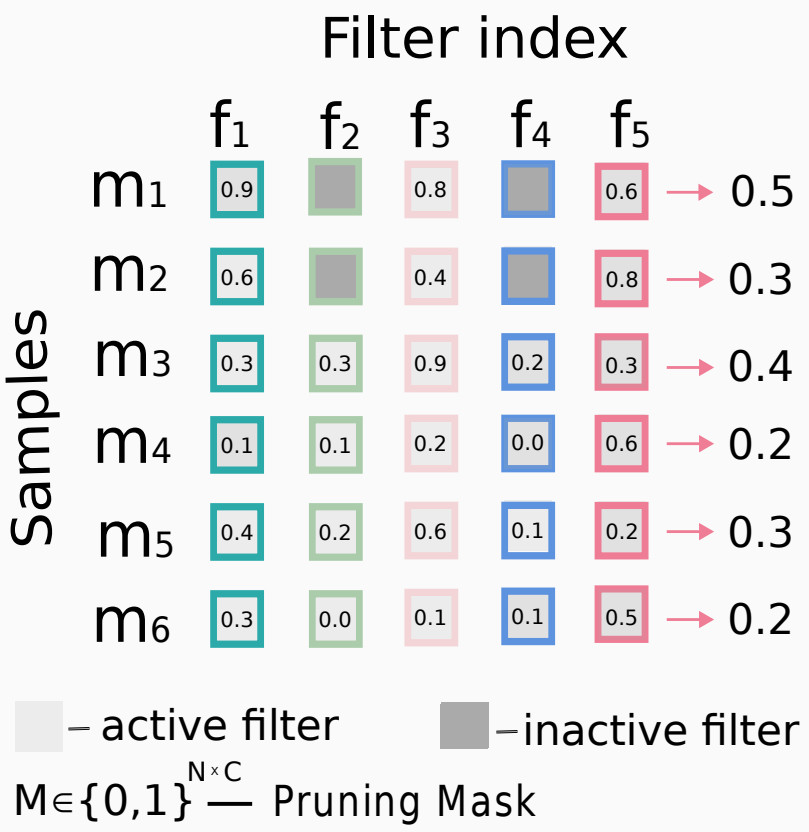
$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- ▷ Instance-wise threshold
- ▷ No manual pruning ratio

Global Consensus Pruning:

$$M[i,j] = \begin{cases} 1 & \text{if } s_{ij} \geq \tau_i \\ 0 & \text{otherwise} \end{cases}$$

- ▷ Prune only if inactive for ALL samples
- ▷ Preserves class-specific features



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

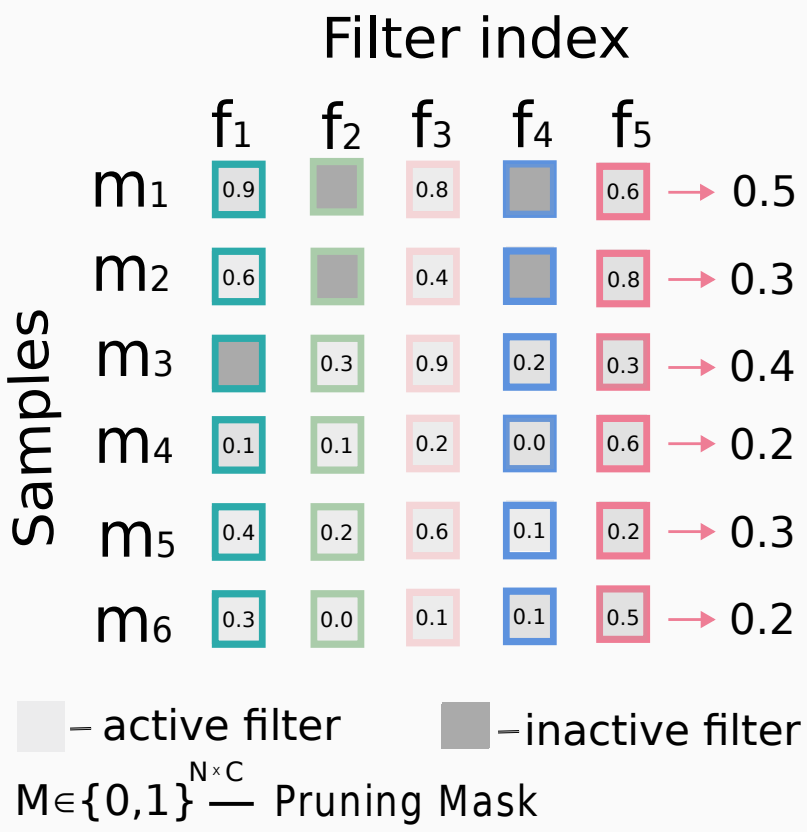
$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- ▷ Instance-wise threshold
- ▷ No manual pruning ratio

Global Consensus Pruning:

$$M[i,j] = \begin{cases} 1 & \text{if } s_{ij} \geq \tau_i \\ 0 & \text{otherwise} \end{cases}$$

- ▷ Prune only if inactive for ALL samples
- ▷ Preserves class-specific features



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

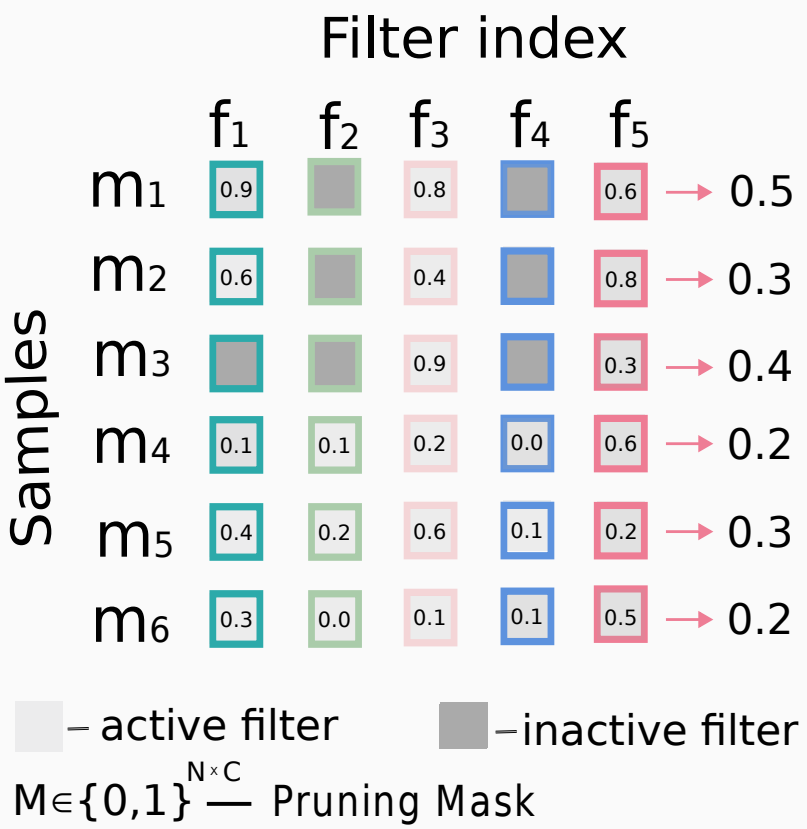
$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- ▷ Instance-wise threshold
- ▷ No manual pruning ratio

Global Consensus Pruning:

$$M[i,j] = \begin{cases} 1 & \text{if } s_{ij} \geq \tau_i \\ 0 & \text{otherwise} \end{cases}$$

- ▷ Prune only if inactive for ALL samples
- ▷ Preserves class-specific features



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

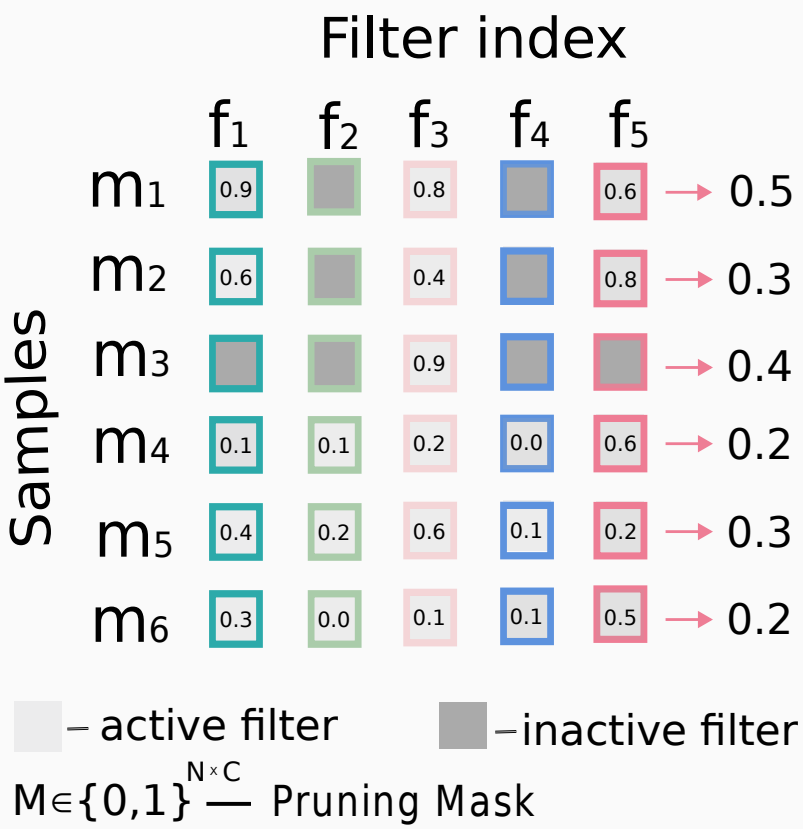
$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- ▷ Instance-wise threshold
- ▷ No manual pruning ratio

Global Consensus Pruning:

$$M[i,j] = \begin{cases} 1 & \text{if } s_{ij} \geq \tau_i \\ 0 & \text{otherwise} \end{cases}$$

- ▷ Prune only if inactive for ALL samples
- ▷ Preserves class-specific features



Global activation analysis identifies redundant filters

Stage 2: Global Activation Analysis & Pruning

Activation Strength:

$$s_{ij} = ||F[i,j,:]||_2 = \sqrt{\sum_{k=1}^{T} |F[i,j,k]|^2}$$

Adaptive Threshold:

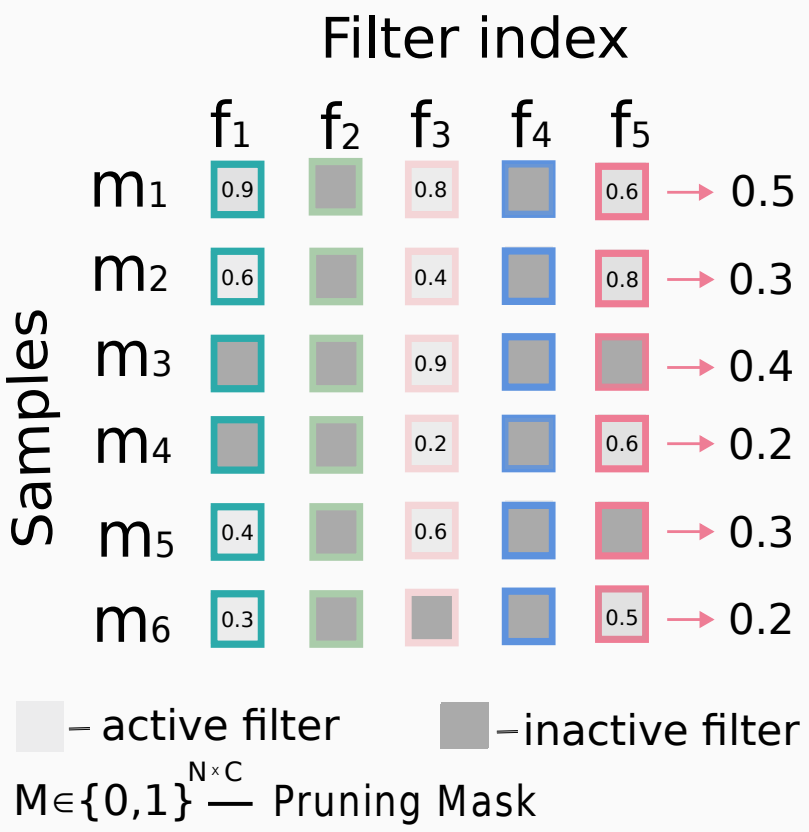
$$\tau_i = \frac{1}{C} \sum_{j=1}^{C} s_{ij}$$

- Instance-wise threshold
- No manual pruning ratio

Global Consensus Pruning:

$$M[i,j] = \begin{cases} 1 & \text{if } s_{ij} \geq \tau_i \\ 0 & \text{otherwise} \end{cases}$$

- Prune only if inactive for ALL samples
- Preserves class-specific features

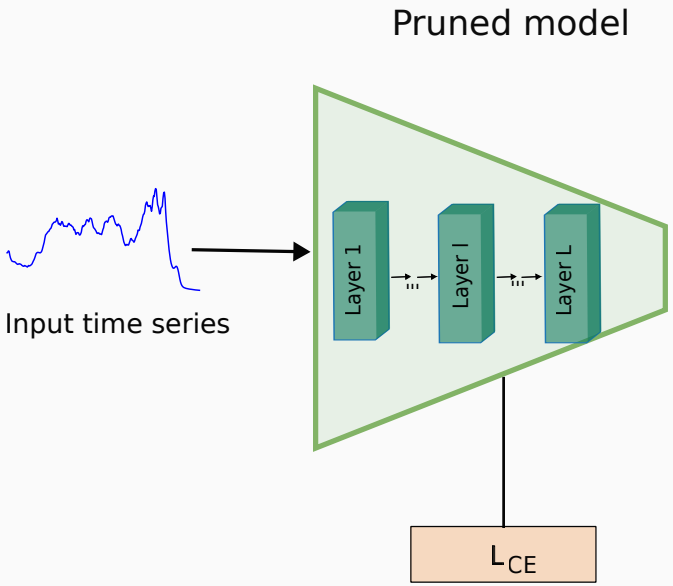


Global activation analysis identifies redundant filters

Stage 3: Retraining

Two Retraining Strategies:

- ▷ **Fine-tuning:** Update from pretrained weights
- ▷ **Scratch training:** Reinitialize and retrain



Scratch training recovers performance after aggressive pruning

1 Introduction and Literature Review

2 Proposed Method

3 Experimental Evaluation

Experimental Setup

- **Base Architectures:** LITE and Inception
  - 5 runs per model, ensemble evaluation
  - LITETime and InceptionTime

Dau et al., *The UCR Time Series Classification Archive*, IEEE/CAA Journal of Automatica Sinica, 2019.

Experimental Setup

- **Base Architectures:** LITE and Inception
  - 5 runs per model, ensemble evaluation
  - LITETime and InceptionTime

- **Datasets:** University California Riverside (UCR) archive 2018
  - 128 univariate time series datasets
  - Diverse domains and characteristics
  - Standard train-test splits, z-normalized

Dau et al., *The UCR Time Series Classification Archive*, IEEE/CAA Journal of Automatica Sinica, 2019.

Experimental Setup

- **Base Architectures:** LITE and Inception
  - 5 runs per model, ensemble evaluation
  - LITETime and InceptionTime
- **Datasets:** University California Riverside (UCR) archive 2018
  - 128 univariate time series datasets
  - Diverse domains and characteristics
  - Standard train-test splits, z-normalized
- **Model Variants Compared**
  - Base models: LITETime and InceptionTime
  - DSP models: Scratch-trained after pruning (DSP-LITETime, DSP-InceptionTime)

Dau et al., *The UCR Time Series Classification Archive*, IEEE/CAA Journal of Automatica Sinica, 2019.

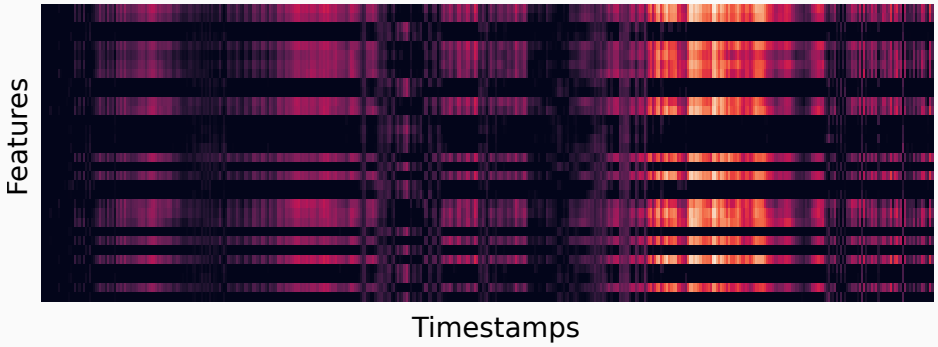
Experimental Setup

- **Base Architectures:** LITE and Inception
  - 5 runs per model, ensemble evaluation
  - LITETime and InceptionTime
- **Datasets:** University California Riverside (UCR) archive 2018
  - 128 univariate time series datasets
  - Diverse domains and characteristics
  - Standard train-test splits, z-normalized
- **Model Variants Compared**
  - Base models: LITETime and InceptionTime
  - DSP models: Scratch-trained after pruning (DSP-LITETime, DSP-InceptionTime)
- **Evaluation Metrics**
  - Classification accuracy (ensemble of 5 runs)
  - Number of wins, ties and losses computed for pairwise comparisons

Dau et al., *The UCR Time Series Classification Archive*, IEEE/CAA Journal of Automatica Sinica, 2019.

Qualitative Redundancy Analysis

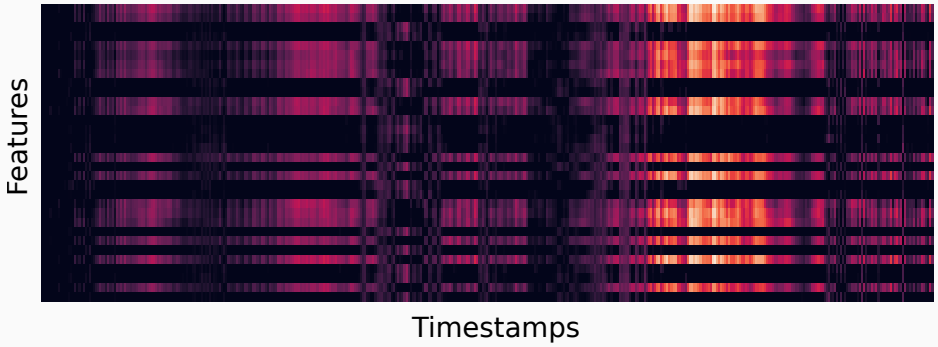
Base LITE Model



- Many active channels
- Redundant patterns

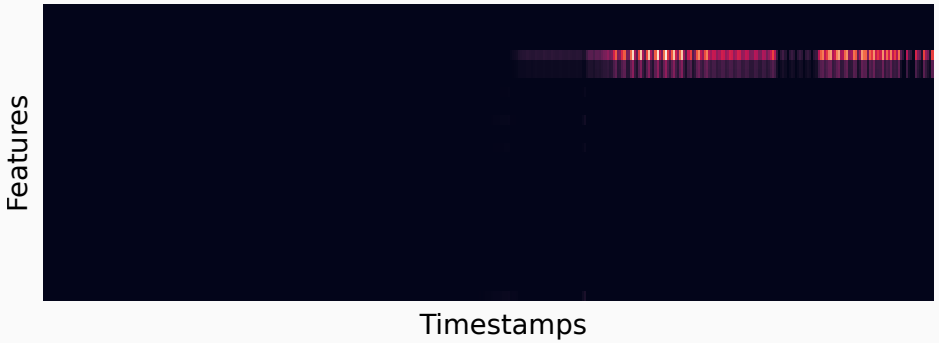
Qualitative Redundancy Analysis

Base LITE Model



- Many active channels
- Redundant patterns

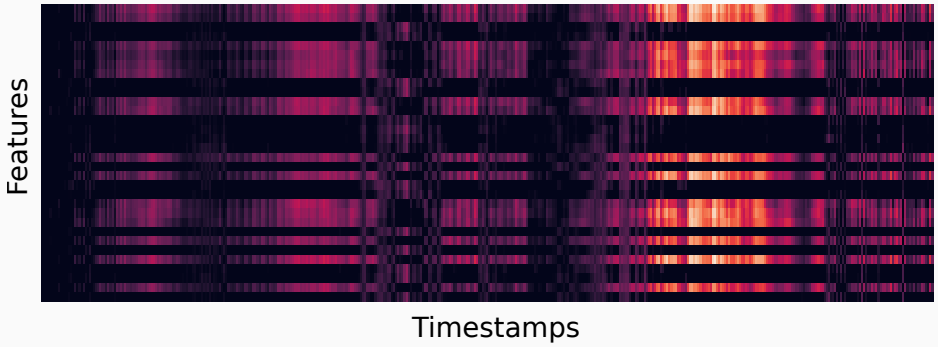
Sparsity-Trained LITE Model



- Compact representation
- Discriminative features

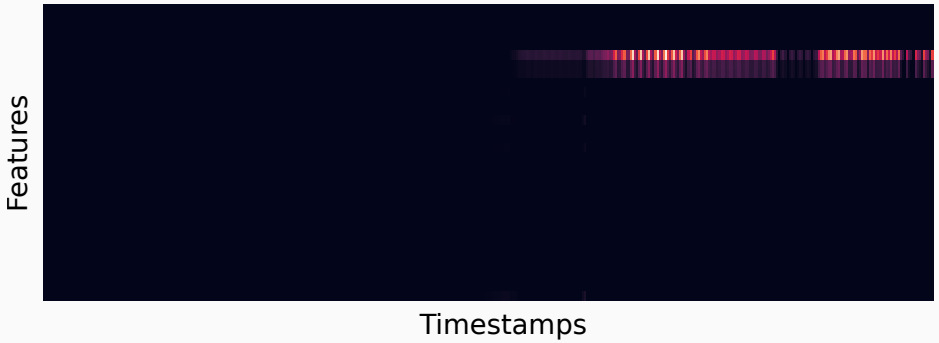
Qualitative Redundancy Analysis

Base LITE Model



- Many active channels
- Redundant patterns

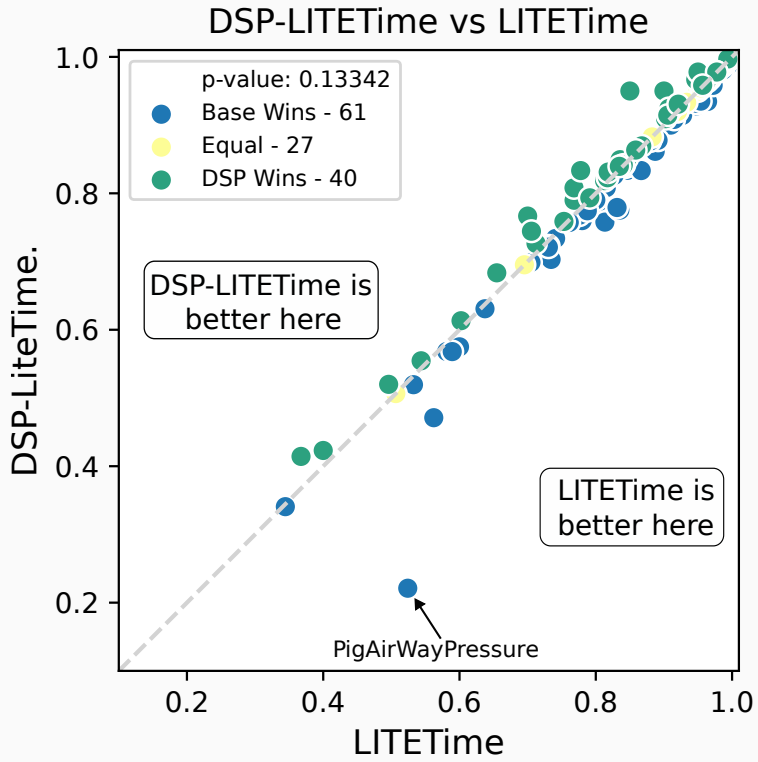
Sparsity-Trained LITE Model



- Compact representation
- Discriminative features

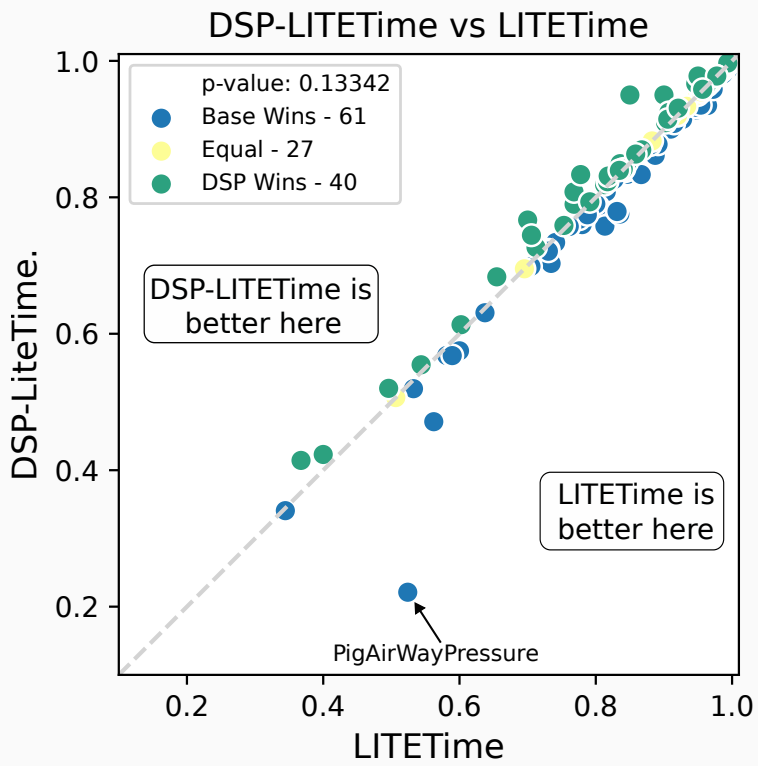
Sparsity loss eliminates redundancy while preserving discriminative patterns

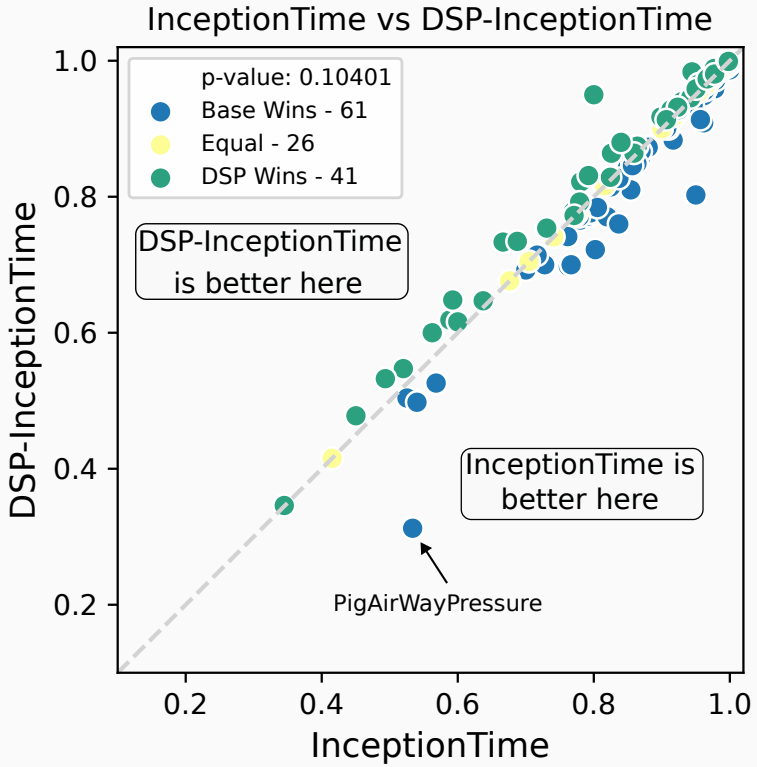
Performance Comparison



- ▷ **58% compression** maintained

Performance Comparison

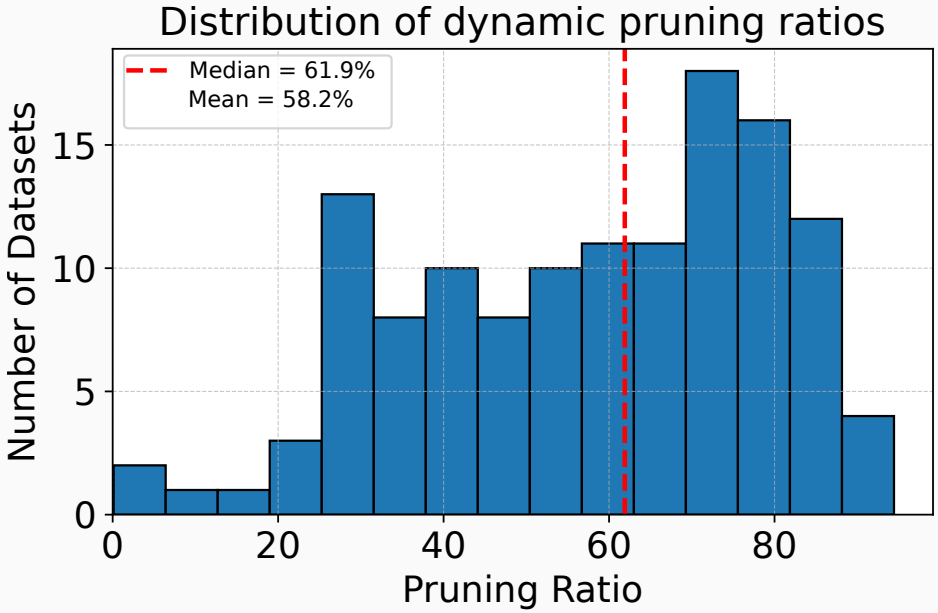




- ▷ **58% compression** maintained
- ▷ **75% compression** maintained

Adaptive Pruning Ratios

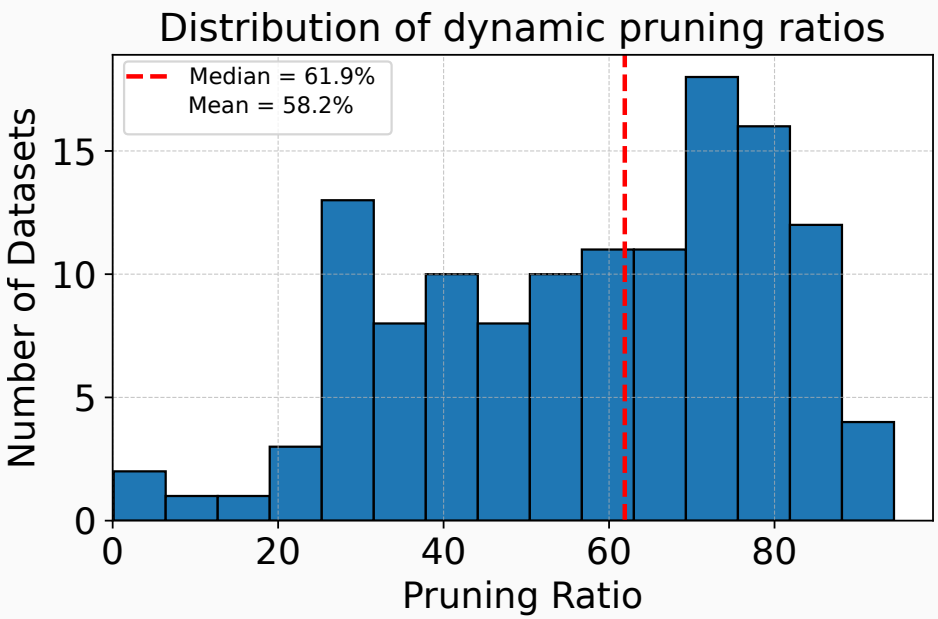
LITETime Model



- Range: 0.01% - 90%

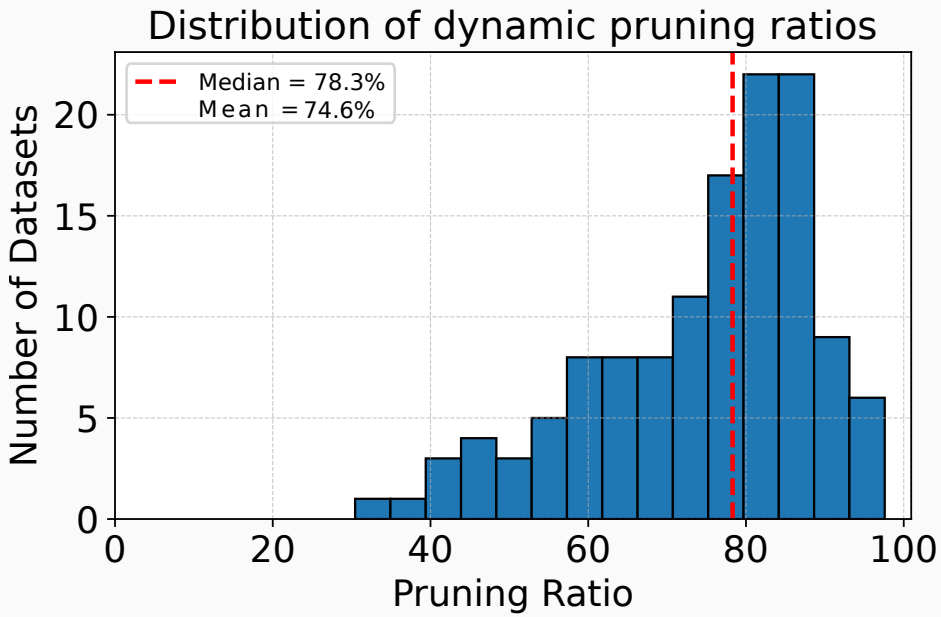
Adaptive Pruning Ratios

LITETime Model



▷ Range: 0.01% - 90%

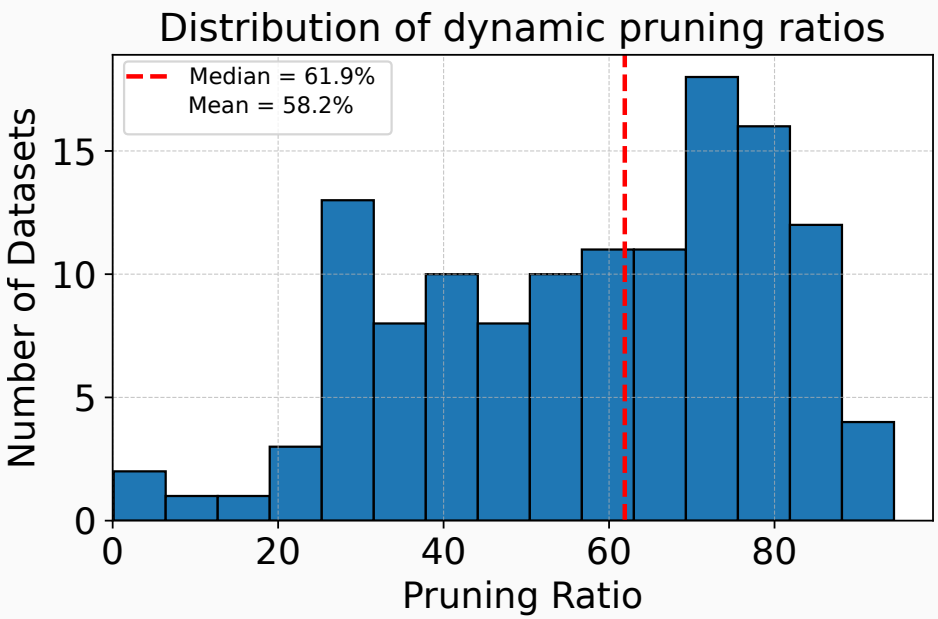
InceptionTime Model



▷ Range: 30.4% - 97.6%

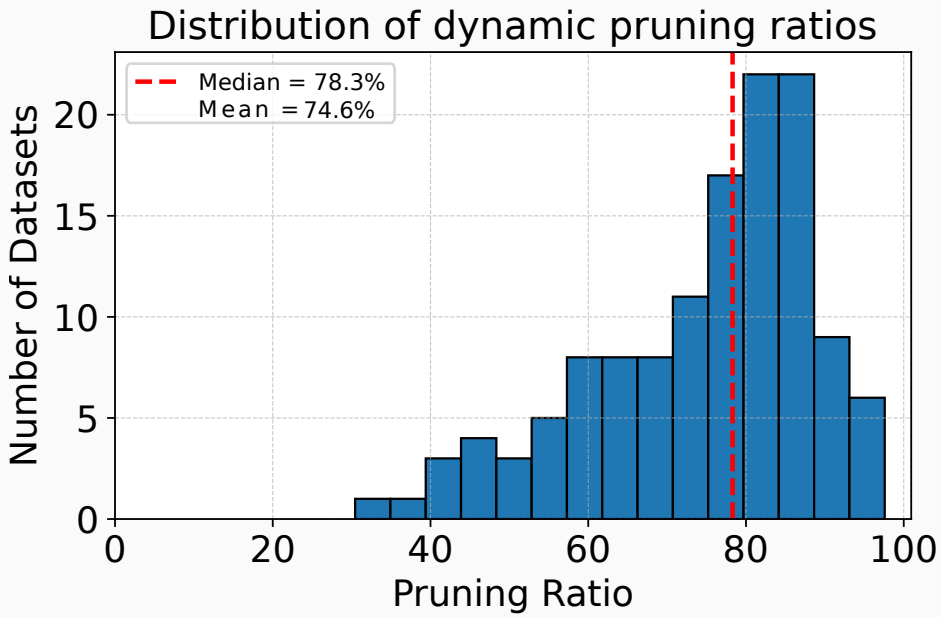
Adaptive Pruning Ratios

LITETime Model



- Range: 0.01% - 90%

InceptionTime Model

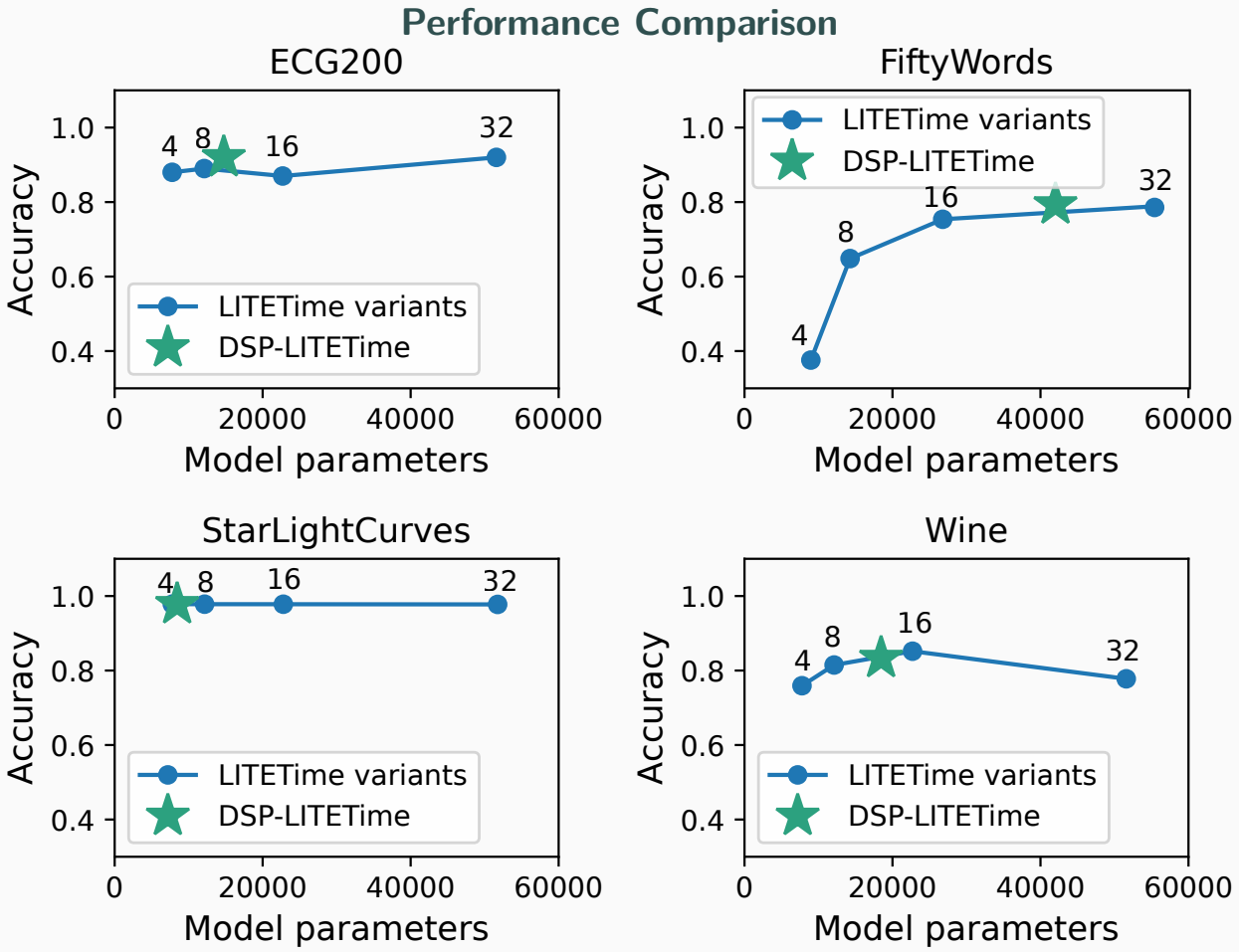


- Range: 30.4% - 97.6%

Key Insights:

- DSP dynamically adapts pruning based on dataset complexity
- Larger model (Inception) → more aggressive pruning
- No manual hyperparameter tuning required

Dynamic vs Static Pruning



Takeaway

- We proposed Dynamic Structured Pruning (DSP), a fully automatic pruning framework for TSC models
- DSP achieves 58% compression for LITETime and 75% for InceptionTime while maintaining accuracy
- No manual hyperparameter tuning required, adapts dynamically to dataset complexity
- Current limitation: validated only on CNN architectures
- Future work: extend to Transformers and combine with quantization and knowledge distillation

Code available here: https://github.com/MSD-IRIMAS/Pruning4TSC

Contact: javidanabdullayev.github.io